Algorithm Complexity

Cost of algorithms — loops and recurrences

✉

Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology

Ho Chi Minh City on April 10, 2026

Outline

1. Why algorithms have a cost
2. Best, average, worst
3. Counting work
4. Asymptotic notation
5. Analysing loops
6. Analysing recurrences
7. Summary

1 Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
Analysing loops
Analysing recurrences
Summary

Why algorithms have a cost

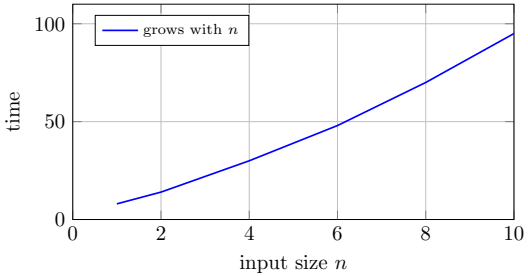
Outline
2 Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
Analysing loops
Analysing recurrences
Summary

Data Structures and Algorithms

- Half the course name is **Algorithms**.
- A correct algorithm is not enough: we must know its **cost**.
- Same problem, many algorithms — we compare running time (and sometimes memory).
- "Easy to code" is software engineering; here we classify **growth with** n.

Outline
3 Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
Analysing loops
Analysing recurrences
Summary

Running Time Is a Function of Input Size



- Write $T(n)$: time as a function of the **input size** n.
- Always say what *n counts*: array length, number of nodes, ...
- One letter, one meaning, on every later slide.

Time vs Extra Space

- **Time:** how the work grows with n.
- **Extra space:** extra memory besides the input.
- Recursion also uses the **call stack** — frames alive *at once*, not the total number of calls.

Outline
5 Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
Analysing loops
Analysing recurrences
Summary

Cost Model



Outline
6 Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
Analysing loops
Analysing recurrences
Summary

Unit-cost RAM

Unless stated otherwise, we use the unit-cost RAM model: primitive operations on fixed-width values take **constant time**.

- Comparisons, assignments, additions, and array accesses take constant time.
- A loop body is constant-time only if every operation and callee inside it is constant-time.
- Arithmetic on arbitrary-precision integers need not be constant-time.
- Later we drop constants and lower-order terms when we *name* a growth class. That does *not* make wall-clock time independent of hardware.

Best, average, worst

Outline
Why algorithms have a cost
7 Best, average, worst
Counting work
Asymptotic notation
Analysing loops
Analysing recurrences
Summary

Three Questions About the Same n

Same algorithm, same n, different inputs:

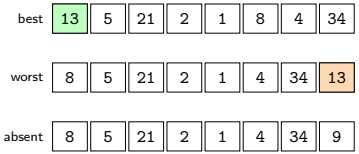
- **Best case** — the most favourable input of size n.
- **Worst case** — the least favourable input of size n.
- **Average case** — expected cost under a **probability model** on the inputs.

This course reports **worst case** unless we say otherwise.

Outline
Why algorithms have a cost
8 Best, average, worst
Counting work
Asymptotic notation
Analysing loops
Analysing recurrences
Summary

Running Example: Sequential Search

Search for K in an **unsorted** array of length n.




```
int seqSearch(int a[], int n,
    int k) {
    for (int i = 0; i < n; ++i)
        if (a[i] == k) return i;
    return -1;
}
```

Best, Average, and Worst for This Search

- **Best:** K is in the first cell — a **constant** number of steps, independent of n.
- **Worst:** K is last, or absent — about n comparisons, **linear** in n.
- **Average** needs a probability model. It is *not* defined as $(\text{best} + \text{worst})/2$.

Assume K is present and **uniformly distributed among the n positions**:

$$E[C] = \frac{1}{n}\sum_{i=1}^{n} i = \frac{n+1}{2},$$

which is still linear in n. If K may be absent, you must also specify $\Pr(K \text{ is present})$.

Algorithm Complexity

Outline

Why algorithms have a cost

10 Best, average, worst

Counting work

Asymptotic notation

Analysing loops

Analysing recurrences

Summary

Experiment vs Theory

Experiment

- Implement, time, plot.
- Depends on hardware, compiler, test data, and system state.
- Gives seconds on *those* runs.

Theory

- Uses a computational model.
- Reasons about all inputs, or a stated input model.
- Abstracts away machine-dependent constant factors.

- Worst-case analysis can guarantee a bound for every input of size n.
- Average-case is meaningful only with a probability model.
- Asymptotics do not tell you the exact number of seconds.

Outline
Why algorithms have a cost
11 Best, average, worst
Counting work
Asymptotic notation
Analysing loops
Analysing recurrences
Summary

Counting work

Outline
Why algorithms have a cost
Best, average, worst
12 Counting work
Asymptotic notation
Analysing loops
Analysing recurrences
Summary

Count Primitive Operations

Assume $n \geq 1$. Count 1 for each array index, comparison, addition, assignment, and return. Do not count reading a simple variable or a literal separately.

```
1  int findMax(int a[], int n) {
2      int currentMax = a[0]; // init: 2 index + assign
3      int i = 1; // init: 1 assign
4      while (i < n) { // tests: n (1 compare each)
5          if (currentMax < a[i]) // body: 2(n-1) index + compare
6              currentMax = a[i]; // body, worst: 2(n-1) index + assign
7          i = i + 1; // updates: 2(n-1) add + assign
8      }
9      return currentMax; // 1
10 }
```


Outline
Why algorithms have a cost
Best, average, worst
13 Counting work
Asymptotic notation
Analysing loops
Analysing recurrences
Summary

The Count, Split by Kind

The body runs $n-1$ times ($i = 1, \ldots, n-1$). The condition is tested n times ($n-1$ true, one false).

Part	What 1 means	Worst-case
`currentMax = a[0]`	index + assign	2
`i = 1`	assign	1
`while (i < n)`	1 compare per test	n
`if (currentMax < a[i])`	index + compare	$2(n-1)$
`currentMax = a[i]`	index + assign	$2(n-1)$
`i = i + 1`	add + assign	$2(n-1)$
`return`	return	1
total $C(n)$	$3 + n + 6(n-1) + 1$	$7n - 2$

- Check: if $n = 1$, the body never runs and $C(1) = 5 = 7 \cdot 1 - 2$.
- Another consistent convention may produce a different exact count, but the growth remains **linear** in n.

Algorithm Complexity
Outline
Why algorithms have a cost
Best, average, worst
14 Counting work
Asymptotic notation
Analysing loops
Analysing recurrences
Summary

From a Count to Running Time

If $C(n)$ is the primitive-operation count, and each primitive costs between two positive constants a and b,

$$a\,C(n) \le T(n) \le b\,C(n).$$

With $C(n) = 7n - 2$ under our convention, $T(n)$ is squeezed between two positive multiples of n: it grows **linearly**.

- Do not replace the number of condition tests by the number of body executions.
- Exact integers depend on the convention; the growth class should not.
- The next section names that class with Big-O, Big-Omega, and Big-Theta.

Algorithm Complexity
Outline
Why algorithms have a cost
Best, average, worst
15 Counting work
Asymptotic notation
Analysing loops
Analysing recurrences
Summary

Constants Do Not Change the Growth Class

- Faster hardware multiplies $T(n)$ by a constant.
- $10^3 n + 10^5$ is still linear; $10n^2 + 10^4 n$ is still quadratic.
- Exact counts matter for constants; the **class** ignores constants and lower-order terms.
- Dropping constants does *not* mean "the machine does not matter in practice".

Outline
Why algorithms have a cost
Best, average, worst
16 Counting work
Asymptotic notation
Analysing loops
Analysing recurrences
Summary

Asymptotic notation

Outline
Why algorithms have a cost
Best, average, worst
Counting work
17 Asymptotic notation
Analysing loops
Analysing recurrences
Summary

Three Symbols, Three Meanings

Write	Read	Role
$T(n) \in O(g(n))$	"no worse than g"	**upper** bound
$T(n) \in \Omega(g(n))$	"at least g"	**lower** bound
$T(n) \in \Theta(g(n))$	"same order as g"	**tight** (both sides)

- These are sets of functions. The set notation $\in$ is the precise one.
- Writing $T(n) = O(g(n))$ is a common shorthand for the same idea.
- Big-O is an upper bound. It is not automatically tight, and it does not mean "worst case".

Algorithm Complexity

Outline

Why algorithms have a cost

Best, average, worst

Counting work

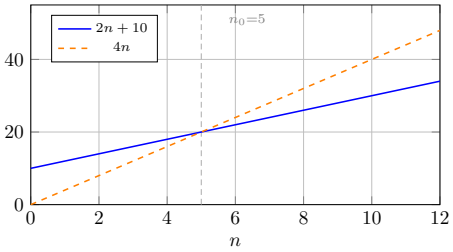
18 Asymptotic notation

Analysing loops

Analysing recurrences

Summary

Big-O Is an Upper Bound

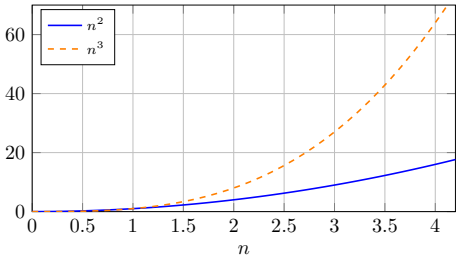


For nonnegative running times, $f(n) \in O(g(n))$ if $\exists\, c > 0$, n_0 such that

$$0 \leq f(n) \leq c\, g(n) \quad \text{for all } n \geq n_0.$$

- $2n + 10 \in O(n)$ (e.g. $c = 4$, $n_0 = 5$).

Big-O Is Not Automatically Tight



- $n^2 \in O(n^2)$ and also $n^2 \in O(n^3)$.
- Both are true: n^3 is a valid **upper** bound, just a loose one.
- You may write O, Ω, or Θ for best, average, or worst case — name the cost function.
- When you know a tight bound, prefer Θ.

Algorithm Complexity

Outline

Why algorithms have a cost

Best, average, worst

Counting work

20 Asymptotic notation

Analysing loops

Analysing recurrences

Summary

Big-Omega Is a Lower Bound

For nonnegative running times, $f(n) \in \Omega(g(n))$ if $\exists\, c > 0$, n_0 such that

$$0 \leq c\, g(n) \leq f(n) \quad \text{for all } n \geq n_0.$$

- Sequential search is $\Omega(n)$ in the worst case.
- $n^2 \in \Omega(n)$ is true; $n^2 \in \Omega(n^3)$ is false.

Algorithm Complexity

Outline

Why algorithms have a cost

Best, average, worst

Counting work

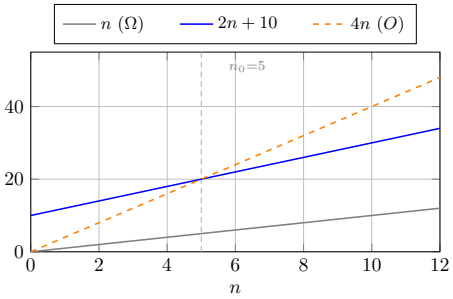
21 Asymptotic notation

Analysing loops

Analysing recurrences

Summary

Big-Theta Is a Tight Bound



$f(n) \in \Theta(g(n))$ if $\exists\, c_1, c_2 > 0$, n_0 such that

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$$

for all $n \geq n_0$.

- Equivalent to $f \in O(g)$ and $f \in \Omega(g)$.
- $2n + 10 \in \Theta(n)$.
- $n^2 \notin \Theta(n^3)$.

Algorithm Complexity

Outline

Why algorithms have a cost

Best, average, worst

Counting work

22 Asymptotic notation

Analysing loops

Analysing recurrences

Summary

How We Write O and Θ

Classroom rule

Keep the **fastest-growing** term. Drop positive **constants** and **slower** terms.

Count	Write
$7n - 2$	$\Theta(n)$
$3n^2 + 100n + 7$	$\Theta(n^2)$
$\frac{n(n+1)}{2} = \frac{n^2}{2} + \frac{n}{2}$	$\Theta(n^2)$

- Same reporting rule for O when you name the standard class: $3n^2 + 100n = O(n^2)$.
- This is how we *write the answer*. The definition remains $\exists\, c, n_0$.
- Do not drop the leading term: $0.001n^2 + 1000n$ is still $\Theta(n^2)$, not $\Theta(n)$.

Algorithm Complexity

Outline

Why algorithms have a cost

Best, average, worst

Counting work

23 Asymptotic notation

Analysing loops

Analysing recurrences

Summary

Classroom Check

Which statements are true?

1. $n^2 \in O(n^3)$
2. $n^2 \in \Theta(n^3)$
3. $n^2 \in \Omega(n)$

True / false / true.

- Prefer Θ when the bound is tight.
- The writing rule is the previous classroom block: drop constants and slower terms, then name Θ.

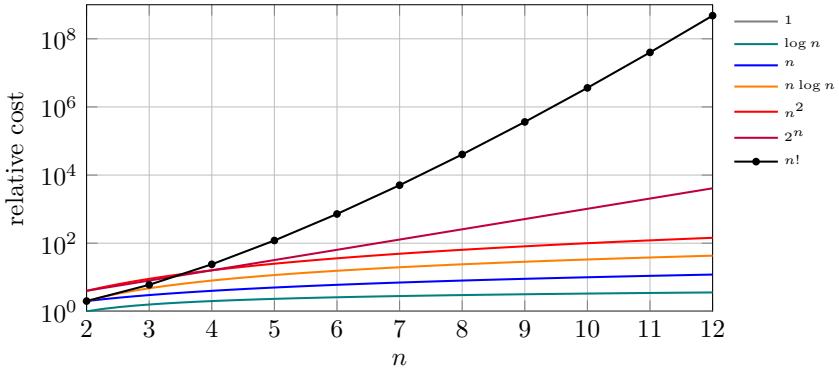
Outline
Why algorithms have a cost
Best, average, worst
Counting work
24 Asymptotic notation
Analysing loops
Analysing recurrences
Summary

Common Growth Classes — Meaning

Class	Typical cause	Feel
$\Theta(1)$	constant $\Theta(1)$ work	constant
$\Theta(\log n)$	index $\times c$ or $/c$ $(c > 1)$	binary search
$\Theta(\sqrt{n})$	$i^2 \leq n$ with $+1$	rarer here
$\Theta(n)$	one linear scan	sequential search
$\Theta(n \log n)$	n outer $\times \log n$ inner	efficient sort
$\Theta(n^2)$	nested $n \times n$ or $\sum i$	naive pairs
$\Theta(2^n)$	two calls of size $n - 1$	Hanoi
$\Theta(\varphi^n)$	naive Fibonacci	overlapping work

Outline
Why algorithms have a cost
Best, average, worst
Counting work
25 Asymptotic notation
Analysing loops
Analysing recurrences
Summary

How Fast They Pull Apart



- Order: $1,\ \log n,\ n,\ n \log n,\ n^2,\ \ldots,\ 2^n,\ n!$.
- $\log_2 n$ and $\log_{10} n$ are the same Θ class.
- Exponential / factorial become unusable for moderate n.

Vertical axis: log scale.

Algorithm Complexity
Outline
Why algorithms have a cost
Best, average, worst
Counting work
26 Asymptotic notation
Analysing loops
Analysing recurrences
Summary

Analysing loops

Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
27 Analysing loops
Analysing recurrences
Summary

A Systematic Recipe for Loops

1. Define the input size n.
2. Identify the operation whose executions will be counted.
3. Determine the values taken by each loop variable.
4. Count the body executions using a product or a summation.
5. Include the cost of operations called inside the body.
6. Simplify the result and report a tight Θ bound.
7. State whether the result is best, average, or worst case.

These loop rules assume that one body execution costs $\Theta(1)$, unless stated otherwise.

Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
28 Analysing loops
Analysing recurrences
Summary

Add Sequential Work; Multiply Independent Loops



Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
29 Analysing loops
Analysing recurrences
Summary

Sequential fragments

$$T(n) = T_1(n) + T_2(n), \qquad \Theta(n) + \Theta(n^2) = \Theta(n^2).$$

Independent nested loops

If the outer loop runs n times and, for each outer iteration, the inner loop runs m times:

$$T(n, m) = \Theta(nm).$$

Write $\Theta(n^2)$ only when both sizes are n.

Dependent Loops Need a Sum

If the inner count depends on the outer variable, do not multiply $n \cdot n$ blindly:

$$T(n) = \sum_{i=1}^{n} C(i).$$

Triangle example:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2).$$

Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
30 Analysing loops
Analysing recurrences
Summary

Mutually Exclusive Branches

For mutually exclusive branches, add the condition and the common work, then take the **maximum** branch cost:

$$T(n) = T_{\text{condition}}(n) + \max\{T_{\text{then}}(n), T_{\text{else}}(n)\}.$$

- This is the worst case of exclusive branches.
- Do not use max if both fragments actually run.

Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
31 Analysing loops
Analysing recurrences
Summary

Case 1 — Constant Iterations: $\Theta(1)$

```
for (int i = 0; i < 10; ++i) {
    work(); // Theta(1)
}
```

- Bound 10 does not depend on n.
- Values of i: $0, 1, \ldots, 9$.
- The body runs exactly 10 times.
- If work() is $\Theta(1)$, then $T(n) = \Theta(1)$.

A fragment with no loop on n is $\Theta(1)$ only if every callee is $\Theta(1)$.

Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
32 Analysing loops
Analysing recurrences
Summary

Case 2 — Linear Scan: $\Theta(n)$

```
for (int i = 0; i < n; i += 2) {
    work(); // Theta(1)
}
```

- Values of i: $0, 2, 4, \ldots$
- Number of iterations: $\lceil n/2 \rceil$.
- $\lceil n/2 \rceil = \Theta(n)$.
- If $\mathrm{work}()$ is $\Theta(1)$, then $T(n) = \Theta(n)$.

A positive constant step $+2$, $+5$, $\ldots$ does not leave the linear class.

Case 3 — Multiply / Divide the Index: $\Theta(\log n)$

```
for (int i = 1; i < n; i
     *= 2) {
    work(); // Theta(1)
}
```



stop when $2^k \geq n$

$\Rightarrow \; k = \lceil \log_2 n \rceil = \Theta(\log n)$

- Values of i: $1, 2, 4, \ldots$
- After k iterations, $i = 2^k$.
- Set up that equation; do not only memorise a slogan.

Algorithm Complexity

Outline

Why algorithms have a cost

Best, average, worst

Counting work

Asymptotic notation

34 Analysing loops

Analysing recurrences

Summary

Case 3 — When the Logarithmic Rule Applies

- The multiplier or divisor must be a constant $c > 1$.
- The loop variable must move toward the stopping condition.
- If the body is not $\Theta(1)$, multiply (or add) the body's cost.
- Do not only memorise "multiply or divide $\Rightarrow$ logarithmic": set up the equation for i.

Case 4 — Square-Root Bound: $\Theta(\sqrt{n})$

```
// Illustration: i*i may overflow
    in C++.
for (int i = 1; i * i <= n; ++i) {
    work(); // Theta(1)
}
```

$$i^2 \le n \iff i \le \sqrt{n}.$$

- Values of i: $1, 2, \ldots, \lfloor \sqrt{n} \rfloor$.
- $\lfloor \sqrt{n} \rfloor$ iterations.
- If $\mathrm{work}()$ is $\Theta(1)$, then $T(n) = \Theta(\sqrt{n})$.

Safer C++ test: $i \le n/i$ (with $i \ge 1$).

Algorithm Complexity

Outline

Why algorithms have a cost

Best, average, worst

Counting work

Asymptotic notation

36 Analysing loops

Analysing recurrences

Summary

Case 5 — Independent Nests: $\Theta(n \log n)$

```
for (int i = 0; i < n; ++i) {
    for (int j = 1; j < n; j *= 2)
        {
        work(); // Theta(1)
    }
}
```

- Outer loop: n iterations.
- For each outer iteration, inner $j = 1, 2, 4, \ldots$ runs $\Theta(\log n)$ times.
- Total $\mathrm{work}()$ calls:

$$n \cdot \Theta(\log n) = \Theta(n \log n).$$

Algorithm Complexity

Outline

Why algorithms have a cost

Best, average, worst

Counting work

Asymptotic notation

37 Analysing loops

Analysing recurrences

Summary

Case 6 — Independent Nests: $\Theta(n^2)$

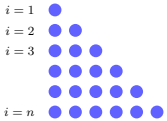
```
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
        work(); // Theta(1)
    }
}
```

- Outer: n iterations; inner: n each time.
- Both sizes are n, so

$$T(n) = \Theta(n \cdot n) = \Theta(n^2).$$

Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
38 Analysing loops
Analysing recurrences
Summary

Case 7 — Dependent Triangle: $\Theta(n^2)$



row i has i cells

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$
$$= \Theta(n^2)$$

```
for (int i = 1; i <= n; ++i) {
    for (int j = 1; j <= i; ++j) {
        work(); // Theta(1)
    }
}
```

- When the outer variable is i, the inner loop runs i times.
- Total:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2).$$

Algorithm Complexity
Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
39 Analysing loops
Analysing recurrences
Summary

Case 8 — Trap: Dependent, but $\Theta(n \log n)$

```
for (int i = 1; i <= n; ++i) {
    for (int j = i; j <= n; j +=
         i) {
        work(); // Theta(1)
    }
}
```

- For fixed i, inner $j = i, 2i, 3i, \ldots$ runs about n/i times.
- Total:

$$T(n) = \sum_{i=1}^{n} \left\lfloor \frac{n}{i} \right\rfloor = \Theta(n \log n).$$

- Dependent nests are not automatically $\Theta(n^2)$.

Algorithm Complexity

Outline

Why algorithms have a cost

Best, average, worst

Counting work

Asymptotic notation

40 Analysing loops

Analysing recurrences

Summary

Case 9 — Trap: the Body Is Not $\Theta(1)$

```
for (int i = 0; i < n; ++i) {
    binSearch(a, 0, n - 1, key); //
        Theta(log n)
}
```

- Outer loop: n iterations.
- Each binSearch is $\Theta(\log n)$ in the worst case.
- Total:

$$n \cdot \Theta(\log n) = \Theta(n \log n).$$

- One loop is not automatically $\Theta(n)$.

Algorithm Complexity

Outline

Why algorithms have a cost

Best, average, worst

Counting work

Asymptotic notation

41 Analysing loops

Analysing recurrences

Summary

Analysing recurrences

Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
Analysing loops
42 Analysing recurrences
Summary

A Systematic Recipe for Recursion

To **write** the recurrence:

1. Size n: what does it count?
2. Base case, and its cost.
3. How many recursive calls in one invocation?
4. Size of each subproblem.
5. Non-recursive work $f(n)$ in the current call.
6. Write $T(n) = \sum T(n_k) + f(n)$.

Then **solve** by unrolling (or a recursion tree), and report time *and* stack depth.

Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
Analysing loops
43 Analysing recurrences
Summary

Case 1 — One Smaller List: Write $T(n)$

n = number of nodes. Code from the programming review.

```
struct Node { int key; Node*
    next; };

int sum(Node* p) {
    if (p == nullptr) return 0;
        // base case
    return p->key +
        sum(p->next); //
        recursive call
}
```

1. Size: n nodes.
2. Base $n = 0$: $\Theta(1)$.
3. Calls: one.
4. Subproblem: $n - 1$ nodes (`p->next`).
5. Local work: $\Theta(1)$ (add `key`).
6. $T(n) = T(n-1) + \Theta(1)$, $T(0) = \Theta(1)$.

Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
Analysing loops
44 Analysing recurrences
Summary

Case 1 — Unroll on $n = 3$

Take $T(n) = T(n-1) + c$ and $T(0) = c$. For a list of three nodes:

$$\begin{aligned} T(3) &= T(2) + c \\ &= (T(1) + c) + c \\ &= (T(0) + c) + 2c \\ &= 4c = \Theta(n). \end{aligned}$$

In general $T(n) = T(0) + nc = \Theta(n)$.

- Time $\Theta(n)$. Recursion depth $\Theta(n)$. Extra stack $\Theta(n)$.
- `fact(n)` is the same recurrence: one call of size $n-1$, local $\Theta(1)$.

Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
Analysing loops
45 Analysing recurrences
Summary

Case 2 — Half the Array: Write $T(n)$

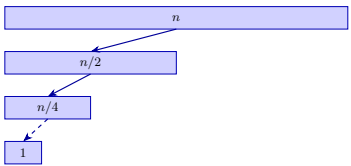
Worst case of recursive binary search. Array already sorted. n = number of remaining cells.

```
int binSearch(int a[], int lo, int hi,
     int k) {
    if (lo > hi) return -1;
    int mid = lo + (hi - lo) / 2;
    if (a[mid] == k) return mid;
    if (k < a[mid])
        return binSearch(a, lo, mid - 1,
             k);
    return binSearch(a, mid + 1, hi, k);
}
```

1. Size: $n = \texttt{hi} - \texttt{lo} + 1$.
2. Base empty / one test: $\Theta(1)$.
3. Calls: one (then or else, not both).
4. Subproblem: $\approx n/2$.
5. Local work: $\Theta(1)$.
6. $T(n) = T(n/2) + \Theta(1)$, $T(1) = \Theta(1)$.

Algorithm Complexity
Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
Analysing loops
46 Analysing recurrences
Summary

Case 2 — Unroll on $n = 8$



one recursive call per level

depth $= \Theta(\log n)$

Assume $n = 8 = 2^3$:

$$8 \to 4 \to 2 \to 1.$$

Three halvings: $\log_2 8 = 3$.

$$T(8) = T(4)+c = T(2)+2c = T(1)+3c = \Theta(\log n).$$

- Worst-case time $\Theta(\log n)$.
  Depth $\Theta(\log n)$.
- Best case: hit at the first mid,
  $\Theta(1)$.

Case 3 — Two Copies of $n-1$: Write $T(n)$

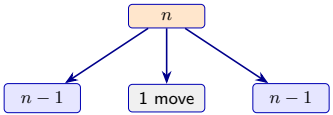
```
void move(int n, char src, char
    dst, char aux) {
    if (n == 1) { printMove(src,
        dst); return; }
    move(n - 1, src, aux, dst);
    move(1, src, dst, aux);
    move(n - 1, aux, dst, src);
}
```

Let $M(n)$ be the number of disk **moves**.

1. Size: n disks.
2. Base $n = 1$: one move, $\Theta(1)$.
3. Calls: two, plus one local move.
4. Both subproblems: $n - 1$.
5. $M(n) = 2M(n-1) + 1$, $M(1) = 1$.

Algorithm Complexity
Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
Analysing loops
48 Analysing recurrences
Summary

Case 3 — Compute $M(1), M(2), M(3), M(4)$



$M(n) = 2M(n-1) + 1, \quad M(0) = 0$

number of moves: $M(n) = 2^n - 1$

running time: $T(n) = \Theta(2^n)$

n	$M(n) = 2M(n-1)+1$	$2^n - 1$
1	1	$2^1 - 1 = 1$
2	$2 \cdot 1 + 1 = 3$	$2^2 - 1 = 3$
3	$2 \cdot 3 + 1 = 7$	$2^3 - 1 = 7$
4	$2 \cdot 7 + 1 = 15$	$2^4 - 1 = 15$

$M(n) = 2^n - 1$, so $T(n) = \Theta(2^n)$.

- Time $\Theta(2^n)$. Depth only $\Theta(n)$.

Case 4 — Two Different Sizes: Write $T(n)$

```
int fib(int n) {
    if (n == 0 || n == 1) return
        n;
    return fib(n - 1) + fib(n -
        2); // overlapping work
}
```

1. Size: the argument n.
2. Base $n = 0$ or 1: $\Theta(1)$.
3. Calls: two.
4. Sizes: $n - 1$ and $n - 2$ — not equal.
5. Local work: $\Theta(1)$.
6. $T(n) = T(n-1) + T(n-2) + \Theta(1)$.

Do **not** write $2T(n-1)$. The code does not memoise.

Algorithm Complexity

Outline

Why algorithms have a cost

Best, average, worst

Counting work

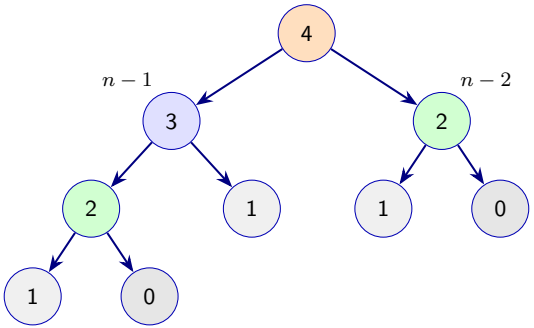
Asymptotic notation

Analysing loops

50 Analysing recurrences

Summary

Case 4 — Expand $\mathrm{fib}(4)$



$\mathrm{fib}(4)$ call tree

same colour ⇒ repeated subproblem

total nodes: exponential
maximum depth: linear

- $T(4) = T(3) + T(2) + c$. The tree has 9 calls; $\mathrm{fib}(2)$ is done twice.
- Each call does $\Theta(1)$ local work, so $T(n) = \Theta(C(n))$ where $C(n)$ is the number of calls.

Algorithm Complexity
Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
Analysing loops
51 Analysing recurrences
Summary

Case 4 — Count Calls, Then Read the Class

$C(n) = 1 + C(n-1) + C(n-2)$, $C(0) = C(1) = 1$. Each call is $\Theta(1)$, so

$$T(n) = \Theta(C(n)).$$

n	$C(n)$	note
0	1	base
1	1	base
2	$1+1+1=3$	
3	$1+3+1=5$	
4	$1+5+3=9$	matches the tree
5	$1+9+5=15$	
6	$1+15+9=25$	

- The sequence grows fast, but the table alone does not yet name the Θ class.
- Next: solve the recurrence, then read φ from both the algebra and the ratios.

Algorithm Complexity
Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
Analysing loops
52 Analysing recurrences
Summary

Case 4 — Solve the Recurrence

Principle. Guess a geometric solution r^n for the homogeneous part $C(n) \approx C(n-1) + C(n-2)$.

$$r^n = r^{n-1} + r^{n-2} \implies r^2 = r + 1 \implies r^2 - r - 1 = 0.$$

The growing root is

$$\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618.$$

(The other root has absolute value < 1 and does not change the Θ class.)

$C(4)/C(3)$	$C(5)/C(4)$	$C(6)/C(5)$	φ
$9/5 = 1.80$	$15/9 \approx 1.67$	$25/15 \approx 1.67$	≈ 1.618

- The extra $+1$ in $C(n)$ is a constant; it does not change $\Theta(C(n))$.
- Therefore $T(n) = \Theta(\varphi^n)$. Faster than every polynomial; slower than Hanoi because $\varphi < 2$.
- Depth follows the $n-1$ chain only: $\Theta(n)$.

Algorithm Complexity

Outline

Why algorithms have a cost

Best, average, worst

Counting work

Asymptotic notation

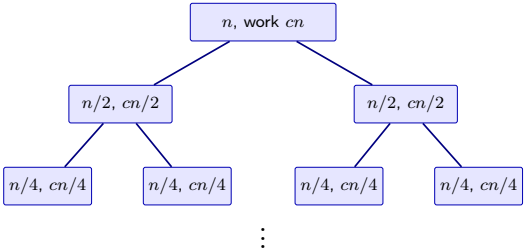
Analysing loops

53 Analysing recurrences

Summary

Case 5 — Two Halves plus Linear Local Work

$$T(n) = 2T(n/2) + \Theta(n), \qquad T(1) = \Theta(1).$$



level 0: cn

level 1: cn

level 2: cn

$\log_2 n$ levels

total $cn \log_2 n = \Theta(n \log n)$

Case 5 — Add the Levels

Assume $n = 8$. Each level of the tree sums to cn:

$$\begin{aligned} \text{level } n: &\quad cn \\ \text{level } n/2: &\quad 2 \cdot (cn/2) = cn \\ \text{level } n/4: &\quad 4 \cdot (cn/4) = cn. \end{aligned}$$

Number of levels: $\Theta(\log n)$. Total: $\Theta(n \log n)$. This is an abstract pattern — not a sorting lecture.

Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
Analysing loops
55 Analysing recurrences
Summary

Time vs Stack — The Cases Together

Case	Recurrence	Time	Depth
list sum / `fact`	$T(n-1)+\Theta(1)$	$\Theta(n)$	$\Theta(n)$
binary search	$T(n/2)+\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
Hanoi	$2T(n-1)+\Theta(1)$	$\Theta(2^n)$	$\Theta(n)$
naive Fib	$T(n-1)+T(n-2)+\Theta(1)$	$\Theta(\varphi^n)$	$\Theta(n)$
two halves $+\Theta(n)$	$2T(n/2)+\Theta(n)$	$\Theta(n\log n)$	$\Theta(\log n)$

- Time is total work. Depth is frames alive at once — not the number of calls.
- Hanoi: $\Theta(2^n)$ calls, only $\Theta(n)$ stack.

Algorithm Complexity
Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
Analysing loops
56 Analysing recurrences
Summary

Summary

Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
Analysing loops
Analysing recurrences
57 Summary

Summary: Loop Patterns

Case	Pattern	Time
1	constant number of iterations	$\Theta(1)$
2	add a positive constant over length n	$\Theta(n)$
3	multiply / divide the index by $c > 1$	$\Theta(\log n)$
4	condition $i^2 \leq n$ with $+1$	$\Theta(\sqrt{n})$
5	independent n and $\log n$ nests	$\Theta(n \log n)$
6–7	independent $n \times n$ / triangular sum	$\Theta(n^2)$
8–9	traps: $\sum n/i$, body not $\Theta(1)$	check, not guess

Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
Analysing loops
Analysing recurrences
58 Summary

Summary: Recurrence Patterns

Case	Pattern	Time
1	$T(n) = T(n-1) + \Theta(1)$	$\Theta(n)$
2	$T(n) = T(n/2) + \Theta(1)$	$\Theta(\log n)$
3	$T(n) = 2T(n-1) + \Theta(1)$	$\Theta(2^n)$
4	$T(n) = T(n-1) + T(n-2) + \Theta(1)$	$\Theta(\varphi^n)$
5	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$

- Patterns are shortcuts, not substitutes for counting. Always inspect the code and state the cost model.
- Prefer Θ when tight. Worst case is the default; average needs a model.
- Separate running time from stack depth.

Outline
Why algorithms have a cost
Best, average, worst
Counting work
Asymptotic notation
Analysing loops
Analysing recurrences
59 Summary